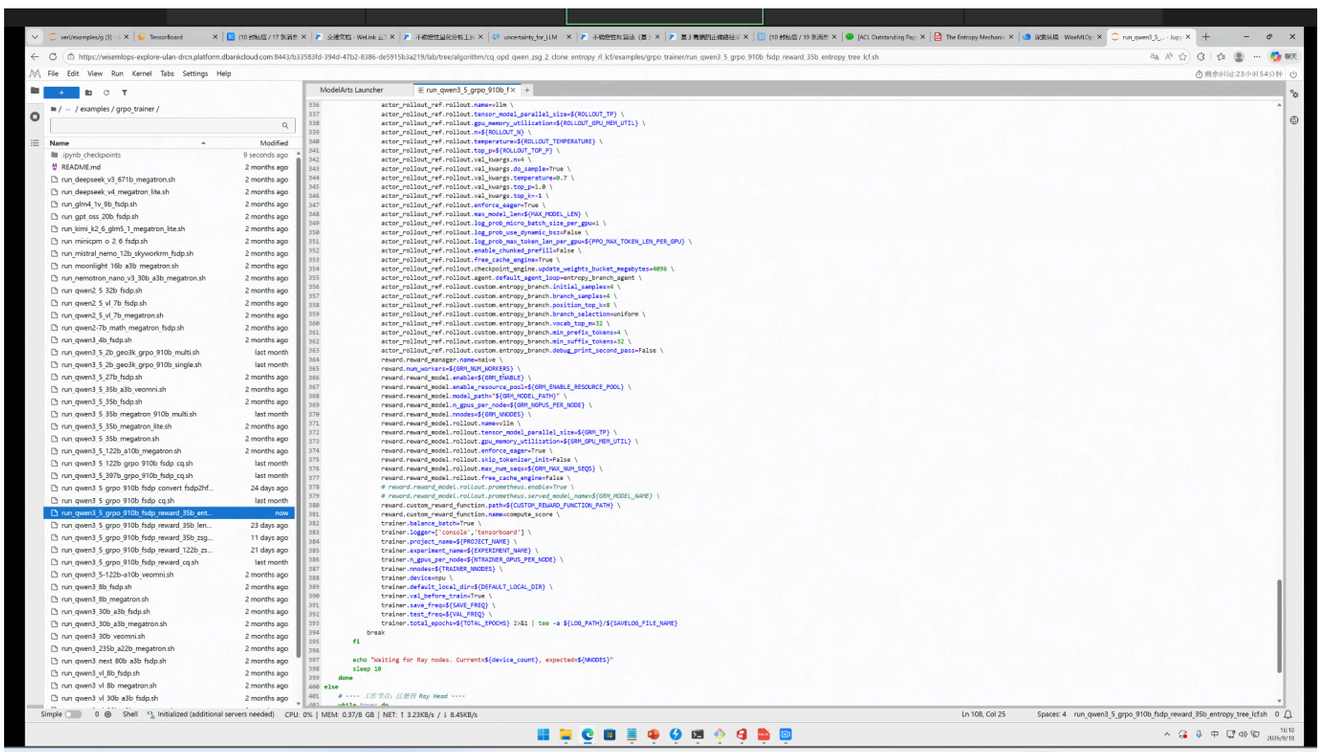
华山平台适配

20260918



9B的模型，输出8192，rollout开thinking模式

```
[ma-user infer]$python3 collect_stats.py --input result_w_thinking_8192.json
Total records              : 812
Records with token length  : 812
Missing token length       : 0
Average output tokens      : 7566.17
Minimum output tokens      : 1217
Median output tokens       : 8192.00
P90 output tokens          : 8192.00
P95 output tokens          : 8192.00
P99 output tokens          : 8192.00
Maximum output tokens      : 8192
Truncated samples          : 602
Truncation denominator     : 812
Truncation rate            : 74.14%
Finish-reason distribution :
  length                       602 (74.14%)
  stop                         210 (25.86%)
```

shuguang设置：35B的模型，rollout也开thinking模式，输出设置2k

verl优化方案

roma上训练7b的耗时统计

阶段	耗时(秒)	占比
update_actor	782.07	79.8%
gen (generate_sequences)	144.68	14.8%
old_log_prob	49.33	5.0%
trajectory_replacement	2.30	0.2%
adv	0.049	~0%
reward	0.00025	~0%
step 总计	979.92	100%

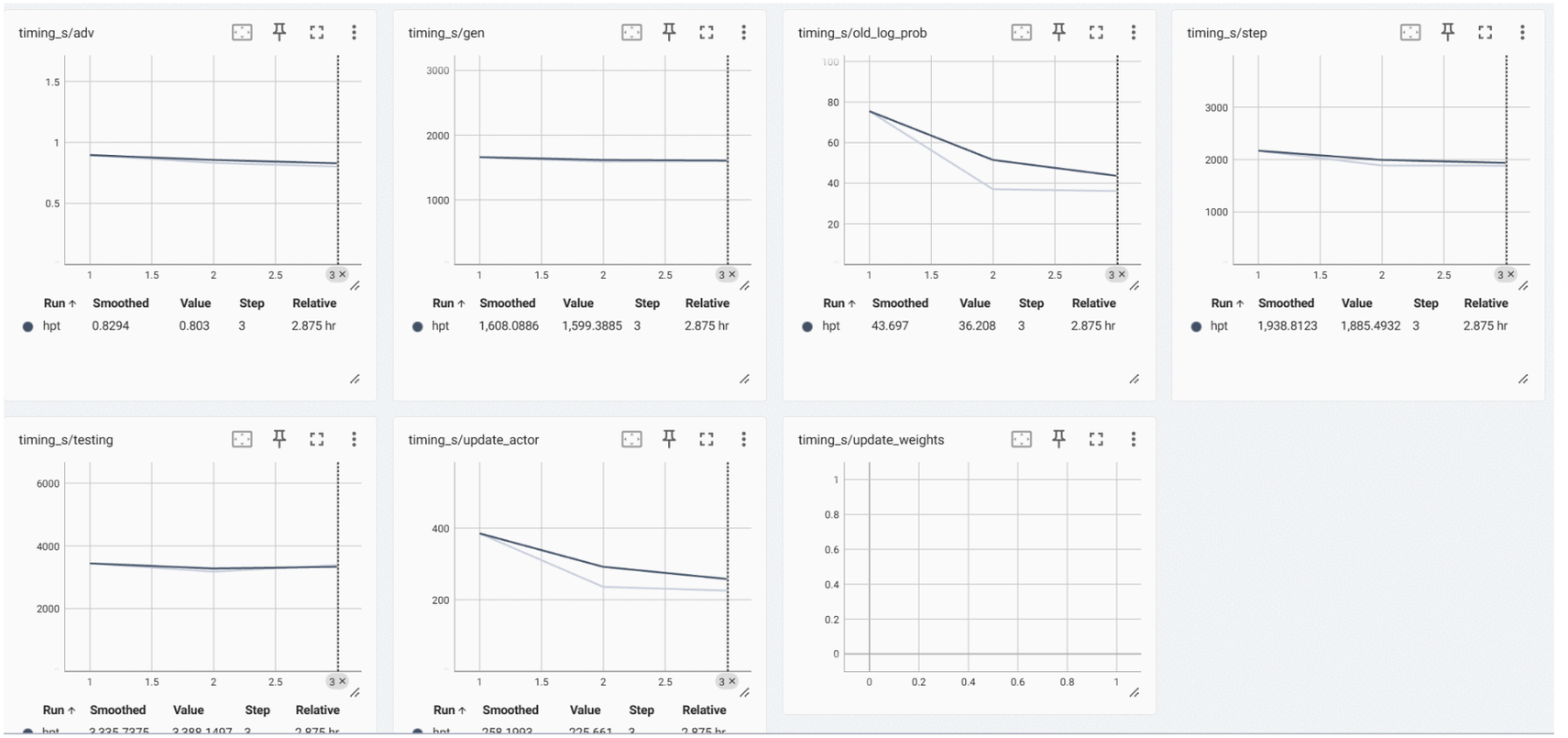
优化建议（按收益排序）：

1. update_actor（最大瓶颈）：这是策略梯度更新阶段，通常包含多次梯度计算和优化器 step。优化方向：
   - 减少更新轮数（如 PPO 的 update_epochs）
   - 减小 batch size 或使用梯度累积
   - 检查是否在 CPU 上计算梯度（应使用 GPU）
   - 使用混合精度训练（AMP/bf16）
   - 检查 update_actor 内部是否有重复的前向/反向传播
2. generate_sequences（推理生成）：144.68 秒。优化方向：
   - 使用 vLLM 等推理加速框架
   - 增大 batch size 提高吞吐
   - 检查 max_new_tokens 是否过大
3. old_log_prob：49.33 秒。这是计算旧策略 log prob 的，可考虑缓存或并行化。

吞吐量：perf/throughput: 116.47 tokens/s，perf/time_per_step: 979.92s，total_num_tokens: 913022。

最值得优先处理的是 update_actor，它占了近 80% 的时间，优化它收益最大。

华山上训练35B模型的耗时统计：validation部分占比2/3，其次是rollout部分耗时最多1h33min一个step，36min是实际训练，接近1h是验证集验证时间



1. 解决oom问题的是去掉下面这个参数：

```python
actor_rollout_ref.actor.entropy_from_logits_with_chunking=True \
```

不是这个愿意，是因为去掉了entropy_loss，entropy_coef=0.0

2. 验证step=20，时间如下：

```
6219 [36m(TaskRunnerV1 pid=796, ip=172.16.139.171)[0m
Training Progress:  0%|          | 0/368 [00:00<?, ?it/s] 6581 [36m(TaskRunnerV1 pid=796, ip=172.16.139.171)[0m
Training Progress:  0%|          | 1/368 [36:33<223:36:58, 2193.51s/it]
```

3. grpo group调小为4，对应mini调成12，

方案一（推荐）：增大 `PPO_MINI_BATCH_SIZE`

当前 `data_parallel_size = 48`，`ROLLOUT_N = 4`，要求：

```
text
```

复制 下载

```
(PPO_MINI_BATCH_SIZE * ROLLOUT_N) % 48 == 0
=> (PPO_MINI_BATCH_SIZE * 4) % 48 == 0
=> PPO_MINI_BATCH_SIZE 必须是 12 的倍数
```

所以把：

```
bash
```

复制 下载

```
PPO_MINI_BATCH_SIZE=${PPO_MINI_BATCH_SIZE:-6}
```

改成：

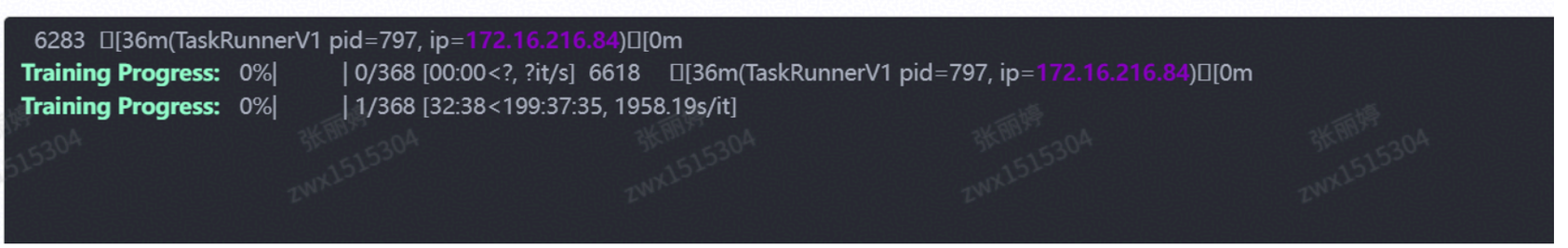
```
bash
```

复制 下载

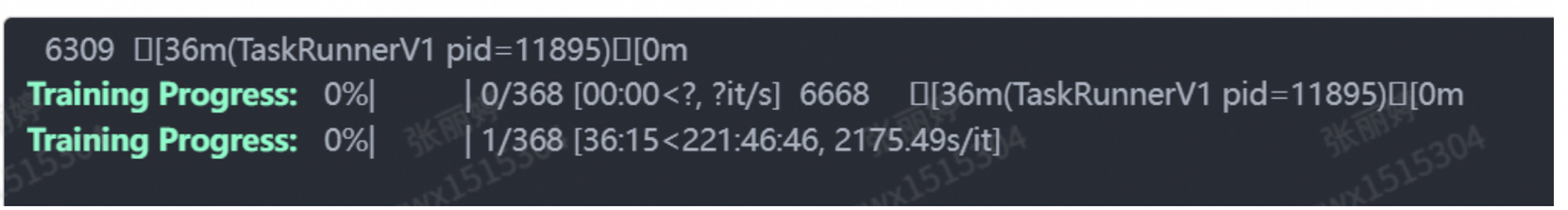
```
PPO_MINI_BATCH_SIZE=${PPO_MINI_BATCH_SIZE:-12}
```

这样有效 `mini_batch_size = 12 * 4 = 48`，`48 % 48 == 0`，每个 DP rank 正好分到 1 个样本。

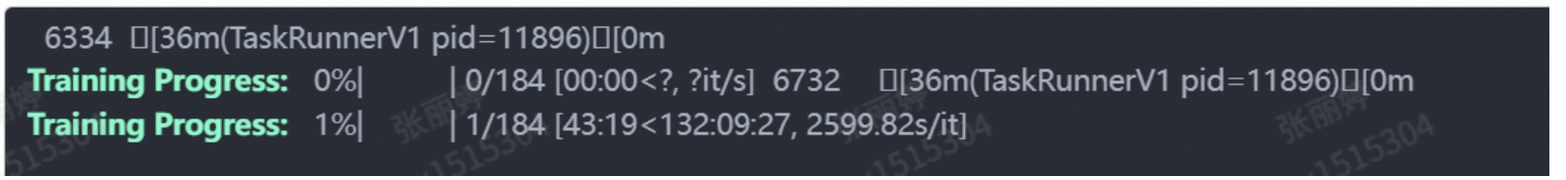
时间如下：



4. group还是8，tp从8调成4

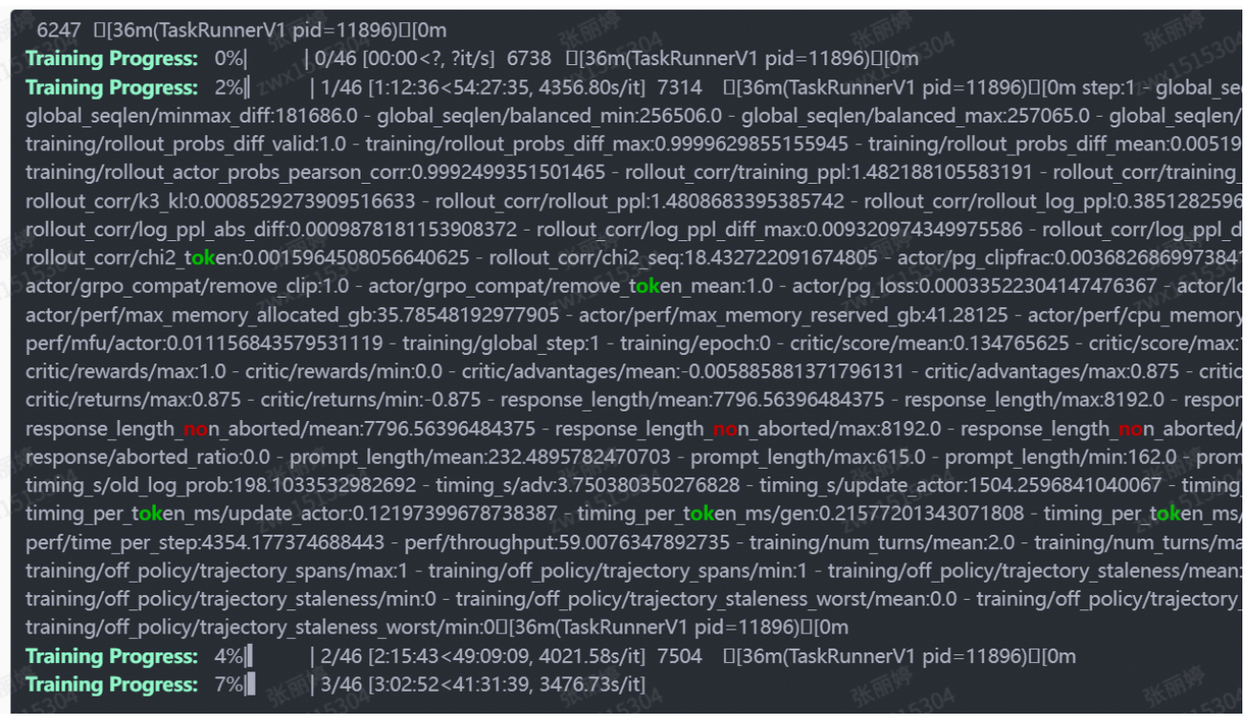


5. tp=4，train_batch_size从48调高到96，



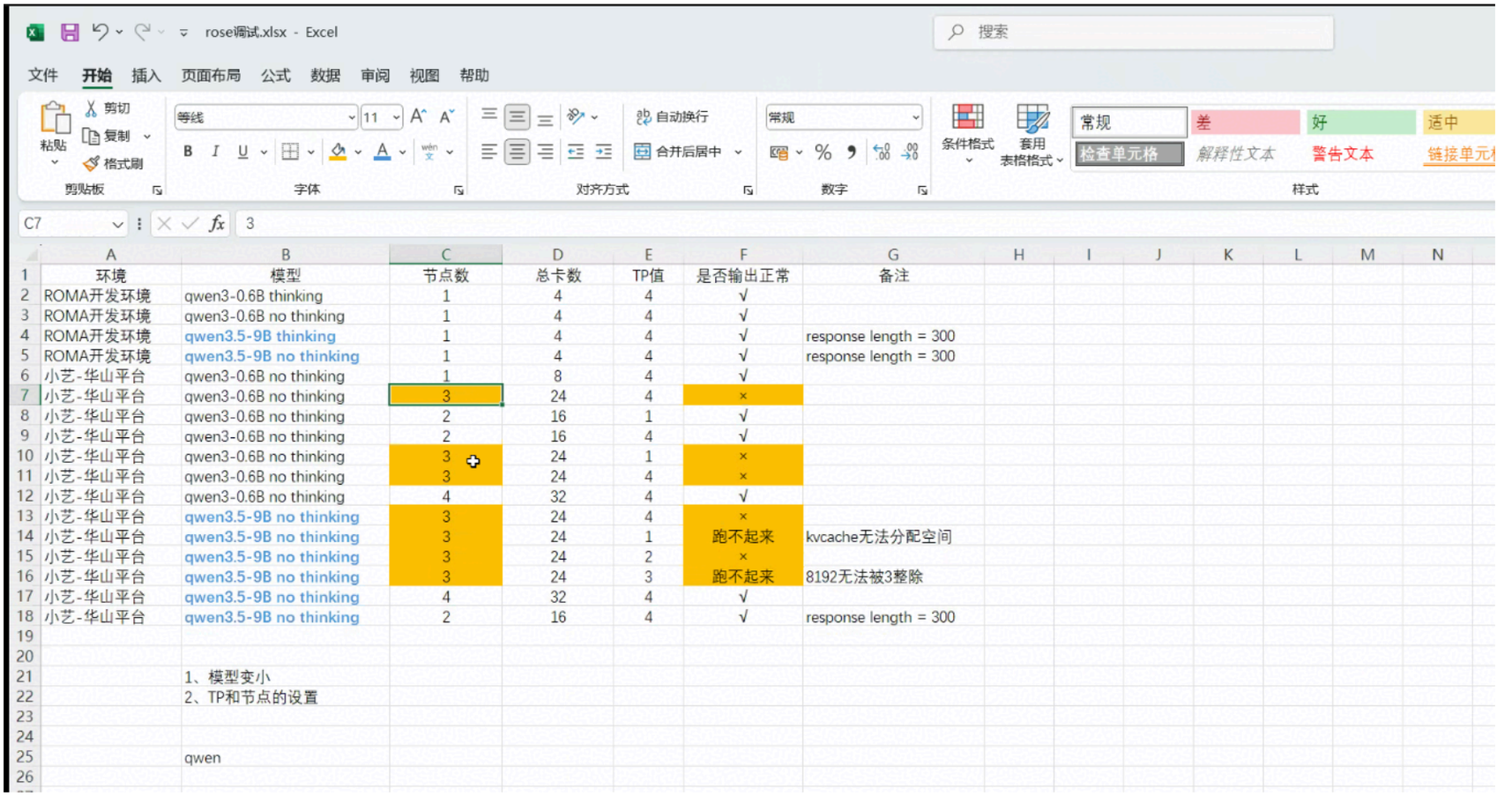
6. tp=4，train_batch_size96，group4

7. train_batch_size=192, tp=4, epoch=1, val_n=1, group=8



hongxin

TP的问题，不是TP的问题，节点数设成3就会出现乱码



	A	B	C	D	E	F	G
1	环境	模型	节点数	总卡数	TP值	是否输出正常	备注
2	ROMA开发环境	qwen3-0.6B thinking	1	4	4	√	
3	ROMA开发环境	qwen3-0.6B no thinking	1	4	4	√	
4	ROMA开发环境	qwen3.5-9B thinking	1	4	4	√	response length = 300
5	ROMA开发环境	qwen3.5-9B no thinking	1	4	4	√	response length = 300
6	小艺-华山平台	qwen3-0.6B no thinking	1	8	4	√	
7	小艺-华山平台	qwen3-0.6B no thinking	3	24	4	×	
8	小艺-华山平台	qwen3-0.6B no thinking	2	16	1	√	
9	小艺-华山平台	qwen3-0.6B no thinking	2	16	4	√	
10	小艺-华山平台	qwen3-0.6B no thinking	3	24	1	×	
11	小艺-华山平台	qwen3-0.6B no thinking	3	24	4	×	
12	小艺-华山平台	qwen3-0.6B no thinking	4	32	4	√	
13	小艺-华山平台	qwen3.5-9B no thinking	3	24	4	×	
14	小艺-华山平台	qwen3.5-9B no thinking	3	24	1	跑不起来	kvcache无法分配空间
15	小艺-华山平台	qwen3.5-9B no thinking	3	24	2	×	
16	小艺-华山平台	qwen3.5-9B no thinking	3	24	3	跑不起来	8192无法被3整除
17	小艺-华山平台	qwen3.5-9B no thinking	4	32	4	√	
18	小艺-华山平台	qwen3.5-9B no thinking	2	16	4	√	response length = 300
19							
20							
21		1、模型变小					
22		2、TP和节点的设置					
23							
24							
25		qwen					
26							

TODO：